

Practice 15 详解答案

A. 单项选择题

1. **B**: 继承让派生类复用并扩展基类。
2. **C**: `Note` 是被 `TextNote` 继承的基类。
3. **B**: 基类函数的 `virtual` 启用运行时多态。
4. **B**: 派生类为继承的虚函数提供新实现叫重写。
5. **B**: 虚函数调用会按对象真实类型分派到 `Dog::speak()`。
6. **B**: 同一基类接口能处理不同派生对象。
7. **B**: 确保 `delete basePtr` 时派生类析构函数也会运行。
8. **B**: 派生类可访问继承来的可访问成员。
9. **B**: 基类指针可指向不同派生类对象。
10. **A**: 共同接口可处理不同的具体对象类型。

B. 填空题

1. 基 (父)
2. 派生 (子)
3. `virtual`
4. 重写 (`override`)
5. 多态

C. 判断题

1. **对**: 派生类可新增自己的状态和行为。
2. **错**: 合理设计的继承可保留 `private` 数据并提供受控接口。
3. **对**: 要经由基类指针/引用进行动态分派, 基类中对应函数必须为虚函数。
4. **错**: `override` 不是强制语法, 但强烈建议使用, 能让编译器检查重写是否正确。
5. **对**: 非虚函数按静态类型绑定。
6. **对**: 这是向上转型, 例如 `Animal* p = new Dog;`。
7. **对**: 这种删除方式是未定义行为, 可能未执行派生类清理工作。
8. **错**: 派生类构造时会先调用基类构造函数, 但不会自动拥有所有同签名构造函数; 需要自己定义或用 `using Base::Base` 继承构造函数。
9. **对**: 虚函数调用的实际行为由对象的真实类型决定。
10. **对**: `Dog is an Animal` 是典型的 is-a 关系。

D. 代码阅读

1. 输出

输出是 `Animal` 。 `Dog` 没有重写 `speak()` ，所以继承并使用 `Animal::speak()` 。

2. 输出

输出是 `Dog` 。虽然变量类型是 `Animal*` ，但它实际指向 `Dog` ，而 `speak()` 是虚函数，因此动态绑定到 `Dog::speak()` 。

3. 代码问题

基类函数名是 `area()` ，派生类写成了 `Area()` ，C++ 区分大小写，因此没有重写同一个函数。 `override` 会让编译器报错。应写成：

```
double area() override;
```

此外，若 `Shape::area()` 没有函数体，通常应写为纯虚函数：

```
virtual double area() const = 0;
```

4. 为什么推荐虚析构函数

```
virtual ~Shape() {}
```

当通过 `Shape*` 删除实际的 `Circle` 、 `Rectangle` 等派生对象时，它会先调用派生类析构函数，再调用 `Shape` 析构函数。多态基类通常写为：

```
virtual ~Shape() = default;
```

5. 展示的原则

```
Shape* shape = new Circle();  
shape->draw();
```

这展示了**运行时多态/动态绑定**：用基类指针表示派生对象，调用虚函数时执行 `Circle` 的实现。

E. 简答题

1. 继承与多态的区别

继承是类之间的关系：派生类复用或扩展基类，表达 is-a。多态是使用方式：通过基类指针或引用调用虚函数时，同一调用会根据对象真实类型执行不同实现。继承提供共同接口，虚函数使该接口产生运行时多态。

2. 为什么虚函数重要

虚函数使调用不依赖指针或引用的静态类型，而依赖对象的实际类型。这让程序可将 Circle、Rectangle 统一作为 Shape 处理，新增派生类时不必修改处理循环。

3. 为什么继承是 is-a 关系

公共继承表示派生对象可在需要基类对象的地方使用。例如每只 Dog 都是 Animal，每个 TextNote 都是 Note。若关系只是“拥有”，例如 Car 有 Engine，则应使用组合而不是继承。

F. 编程题

1. 第一个继承层次

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void eat() const
    {
        cout << "Animal is eating.\n";
    }
};

class Dog : public Animal
{
public:
    void bark() const
    {
        cout << "Woof!\n";
    }
};

int main()
{
    Dog dog;
    dog.eat();
    dog.bark();
}
```

Dog 从 Animal 继承公开的 eat()，同时增加自己的 bark()。

2. 函数重写

```
#include <iostream>
using namespace std;

class Employee
{
public:
    virtual void work() const
    {
        cout << "Employee is working.\n";
    }

    virtual ~Employee() = default;
};

class Programmer : public Employee
{
public:
    void work() const override
    {
        cout << "Programmer is writing code.\n";
    }
};

int main()
{
    Programmer programmer;
    Employee* employee = &programmer;
    employee->work();
}
```

输出 Programmer is writing code. , 因为指针真实指向 Programmer 。

3. Shape 层次结构

```
#include <iostream>
using namespace std;

class Shape
{
public:
    virtual void draw() const
    {
        cout << "Drawing a shape.\n";
    }

    virtual ~Shape() = default;
};

class Circle : public Shape
{
public:
    void draw() const override
    {
        cout << "Drawing a circle.\n";
    }
};

class Rectangle : public Shape
{
public:
    void draw() const override
    {
        cout << "Drawing a rectangle.\n";
    }
};

int main()
{
    Circle circle;
    Rectangle rectangle;
    Shape* shapes[] = {&circle, &rectangle};

    for (Shape* shape : shapes) {
        shape->draw();
    }
}
```

数组统一保存 Shape*，循环中分别调用 Circle::draw() 和 Rectangle::draw()。

4. 虚析构函数练习

```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual ~Base()
    {
        cout << "Base destroyed.\n";
    }
};

class Derived : public Base
{
public:
    ~Derived() override
    {
        cout << "Derived destroyed.\n";
    }
};

int main()
{
    Base* object = new Derived();
    delete object;
}
```

输出顺序：

```
Derived destroyed.
Base destroyed.
```

对象销毁时先清理派生部分，再清理基类部分。

5. CLI Notes 多态 (挑战)

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

using namespace std;

class Note
{
public:
    virtual void display() const = 0;
    virtual ~Note() = default;
};

class TextNote : public Note
{
private:
    string text;

public:
    TextNote(const string& noteText) : text(noteText) {}

    void display() const override
    {
        cout << "[Text] " << text << '\n';
    }
};

class TodoNote : public Note
{
private:
    string task;
    bool completed;

public:
    TodoNote(const string& todoTask, bool isCompleted)
        : task(todoTask), completed(isCompleted) {}

    void display() const override
    {
        cout << "[Todo] " << (completed ? "[x] " : "[ ] ") << task << '\n';
    }
};
```



```

class ReminderNote : public Note
{
private:
    string reminder;
    string date;

public:
    ReminderNote(const string& reminderText, const string& reminderDate)
        : reminder(reminderText), date(reminderDate) {}

    void display() const override
    {
        cout << "[Reminder: " << date << "]" << reminder << '\n';
    }
};

int main()
{
    vector<unique_ptr<Note>> notes;
    notes.push_back(make_unique<TextNote>("Review virtual functions."));
    notes.push_back(make_unique<TodoNote>("Finish practice 15", false));
    notes.push_back(make_unique<ReminderNote>("Team meeting", "Friday"));

    for (const unique_ptr<Note>& note : notes) {
        note->display();
    }
}

```

`unique_ptr<Note>` 自动释放各派生对象，避免手写 `delete`；循环只依赖 `Note` 接口。

G. 设计与思考挑战

设计 B 更好。设计 A 的 `display()` 必须用不断增长的 `if/else if` 判断类型；每新增一种笔记都要修改原函数，容易遗漏且违背“对扩展开放、对修改封闭”。设计 B 将显示行为放到各派生类，新增 `ImageNote` 时只需新增一个类并实现 `display()`；现有遍历 `Note*` 的代码无需修改。各笔记类型的状态和行为也更集中。